

CSCI 383 Exam 1 Review

Regular languages

Fall 2026

Exam format

Three multipart questions

50 minutes

Open notes

- You may print out sections of Kozen and bring them if you wish; I'll note having the book is rarely helpful on exams as you spend more time looking through it than solving the problems
- I recommend writing down the key ideas from the book instead (and understanding them)

No computers

New this year: Proctoring 🙄

Potential question types

True/false questions with 2-sentences max explanation/counter example

- These are designed to test your understanding of the definitions and mechanics of DFAs, NFAs, and regular expressions

Constructions

- E.g., Create a DFA/NFA/regex for a language

Proofs

- Prove the class of regular languages is closed under an operation (i.e., if A is regular, prove that $f(A)$ is regular for some operation on languages)
- Prove that a language is not regular (pumping lemma or closure properties)

DFA

A DFA is a 5-tuple: $M = (Q, \Sigma, \delta, s, F)$

- Q : Finite, nonempty set of sets
- Σ : Finite, nonempty
- $\delta : Q \times \Sigma \rightarrow Q$
- $s \in Q$: Start state
- $F \subseteq Q$: Set of accepting states
- $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$
 - $\hat{\delta}(q, \varepsilon) = q$
 - $\hat{\delta}(q, xc) = \delta(\hat{\delta}(q, x), c)$ for $c \in \Sigma$
- M accepts x iff $\hat{\delta}(s, x) \in F$

NFA

An NFA is a 5-tuple: $N = (Q, \Sigma, \Delta, S, F)$

- Q : Finite, nonempty set of sets
- Σ : Finite, nonempty
- $\Delta : Q \times \Sigma \rightarrow P(Q)$
- $S \subseteq Q$: Start state
- $F \subseteq Q$: Set of accepting states
- $\hat{\Delta} : P(Q) \times \Sigma^* \rightarrow P(Q)$
 - $\hat{\Delta}(A, \varepsilon) = A$
 - $\hat{\Delta}(A, xc) = \bigcup_{q \in \hat{\Delta}(A, x)} \Delta(q, c)$ for $c \in \Sigma$
- N accepts x iff $\hat{\Delta}(S, x) \cap F \neq \emptyset$

NFAs with epsilon

We can add ϵ -transitions to our NFAs

The NFA is allowed to follow ϵ -transitions without reading any input

We proved that NFAs with ϵ are no more powerful than NFAs (and thus DFAs)

Regular expressions

Three base cases

1. $\underline{\emptyset}$ $L(\underline{\emptyset}) = \emptyset$

2. $\underline{\varepsilon}$ $L(\underline{\varepsilon}) = \{\varepsilon\}$

3. $\underline{c}$ $L(\underline{c}) = \{c\}$ for any symbol c

Three recursive cases

4. $\underline{\alpha\beta}$ $L(\underline{\alpha\beta}) = L(\underline{\alpha})L(\underline{\beta})$

5. $\underline{\alpha + \beta}$ $L(\underline{\alpha + \beta}) = L(\underline{\alpha}) \cup L(\underline{\beta})$

6. $\underline{\alpha^*}$ $L(\underline{\alpha^*}) = L(\underline{\alpha})^*$

Standard construction techniques

Product construction

- We can combine multiple DFAs into a single DFA by taking the cartesian product of states; e.g., $Q = Q_1 \times Q_2$
- This lets us simulate multiple DFAs at once

Multiple copies of a single set of states (think the typo problem)

- Given a DFA, we can construct a new finite automata by making multiple copies of states; e.g., $Q = Q_1 \times \{1,2\}$ so states become pairs $(q, 1)$ or $(q, 2)$

Standard construction techniques

Power set

- To simulate an NFA's operation on a string, we construct a DFA whose states are the power set of the NFAs
- The DFA keeps track of all possible states the NFA could be in at any given time
- Also useful for problems like Pangram where you need to keep track of a collection of booleans (e.g., have we seen an *a*? Have we seen a *b*?). The states become the set of booleans which are true

Review questions

1. Can an alphabet contain zero symbols?
2. Can an alphabet contain infinitely many symbols?
3. Can a string contain zero symbols?
4. Can a string contain infinitely many symbols?
5. Can a language contain zero strings?
6. Can a language contain infinitely many strings?

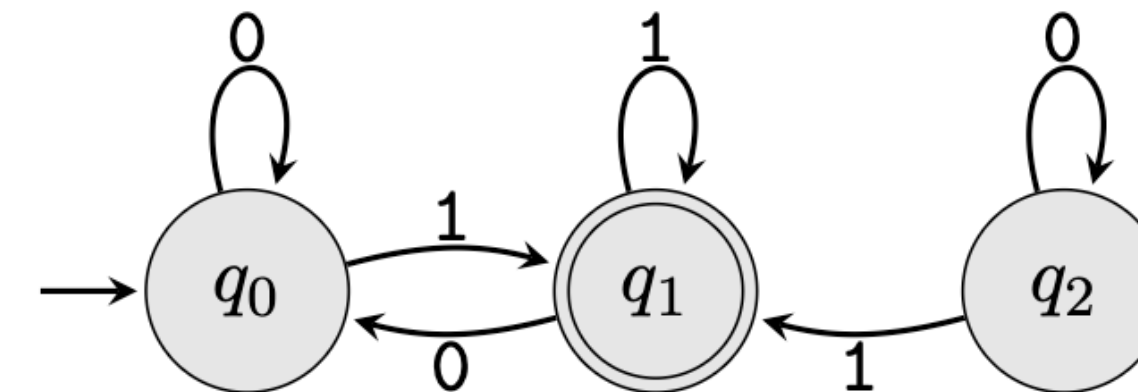
Review questions

7. Can a DFA have 0 states?
8. Can a DFA have infinitely many states?
9. Can the set of accepting states be empty?
10. Can every state in a DFA be an accepting state?
11. Can a DFA have multiple start states?
12. Can an NFA have multiple start states?

Review questions

13. Can a DFA have a state without any transitions out of it?

14. Does this DFA accept 1101?



15. $A = \{w \mid w \in \{0,1\}^* \text{ ends in } 11\}$

The DFA accepts every string in A . Is its language A ?

Proof technique

Structural induction

When proving a fact about $\hat{\delta}$, it's useful to use structural induction

Base case is usually when $x = \varepsilon$

For the inductive step, let $x = ya$, assume the fact about $\hat{\delta}$ is true when $\hat{\delta}$ is applied to y and then prove that it's true for x

Proof technique

Closure properties

Use the closure properties of regular languages to prove that languages are or are not regular

If you can write a language as, for example, the intersection of two regular languages, then the language is regular

If you have a language A you want to show is **not** regular, then if you, for example, take the intersection of A with a regular language B and the resulting language is not regular, then A is not regular

Proof technique

Set equality

You have constructed some finite automaton M and you want to prove that its language is A ; i.e., you want to show $L(M) = A$

Three options:

- Prove a chain of if-and-only-if statements [Usually the shortest proof]:

$$x \in L(M) \iff \dots \iff x \in A$$

Give a justification for each step like [def. of $\hat{\delta}$] or [def. of F]

- Prove two chains of implications:

$$x \in L(M) \implies \dots \implies x \in A$$

$$x \in A \implies \dots \implies x \in L(M)$$

- As a variant on the second approach, use the contrapositive of the second chain of implications

$$x \in L(M) \implies \dots \implies x \in A$$

$$x \notin L(M) \implies \dots \implies x \notin A$$

Closure properties of regular languages

The class of regular languages is closed under

- Union
- Concatenation
- Kleene star
- Intersection
- Complement
- Reversal

Pumping lemma for regular languages

In its contrapositive form!

“Demon game” You want to prove a language A is not regular and the demon is trying to convince you it is regular

1. Demon picks an integer k (the pumping length)
2. You pick strings x, y, z such that $xyz \in A$, and $|y| \geq k$
3. Demon picks strings u, v, w such that $uvw = y$ and $|v| > 0$
4. You pick i such that $xuv^i wz \notin A$

If you can find such an i for any choices the demon makes, you win the game: A is not regular

Review proof

If A is a language over alphabet Σ and $c \in \Sigma$, define $A/c = \{x \mid xc \in A\}$

For example, $\{a^n b^n \mid n \geq 0\} / b = \{a^{n+1} b^n \mid n \geq 0\}$ and
 $\{a^n b^n \mid n \geq 0\} / a = \emptyset$

Prove that if A is a regular language, then A/c is regular.

Hint: Let $M = (Q, \Sigma, \delta, s, F)$ be a DFA whose language is A . Construct a new DFA $M' = (Q', \Sigma, \delta', s', F')$ whose language is A/c . Prove that
 $x \in L(M') \iff x \in A/c$

Review proof

Pumping lemma

Prove that the language $A = \{xxx \mid x \in \{a, b\}^*\}$ is not regular using the pumping lemma

Review proof

Pumping lemma

Prove that the language of palindromes over $\Sigma = \{a, b\}$ is not regular

That is, prove $B = \{x \mid x = x^R\}$ is not regular

Review proof intermission

Give a regular expression that generates the language $C = \{a^m b^n \mid m, n \geq 0\}$

Review proof

Closure properties

Prove that $D = \{a^m b^n \mid m \neq n\}$ is not regular

Hint: Use closure properties of regular languages and the fact that $C = L(\underline{a^* b^*})$ is regular.